

实验一 快速入门 — 摇一摇登录 bupt 校园网

1 实验目的

1. 了解 App Inventor 2 开发环境；
2. 理解 App Inventor 2 开发的基本特点；
3. 熟悉 App Inventor 2 开发过程；
4. 掌握创建项目、设计界面、编写代码块、测试调试、打包应用等基本开发流程和操作；
5. 了解 App Inventor 2 中基本组件（控制、逻辑、文本、变量、过程、事件处理程序、按钮等用户界面组件、界面布局、多媒体音效、加速度传感器等）的用法；

2 实验环境

1. 硬件环境
 - PC 微机（由于在机房电脑上编写和调试，部分实验截图可能不全）
2. 软件环境
 - Windows 操作系统
 - App Inventor2018 离线版（AppInventor2018PersonalEdition_Win）桌面版 AI 伴侣（AI2Companion_Win7_64bit）
3. Android 手机和安装在手机上的 MIT AI2 Companion App。

3 实验要求

1. 创建一个登录北邮校园网的实用 App；
2. 基本功能：支持登录、注销、显示登录状态 3 个基本操作；
3. 实现摇一摇手机自动登录；
4. 登录和注销时有提示音效；

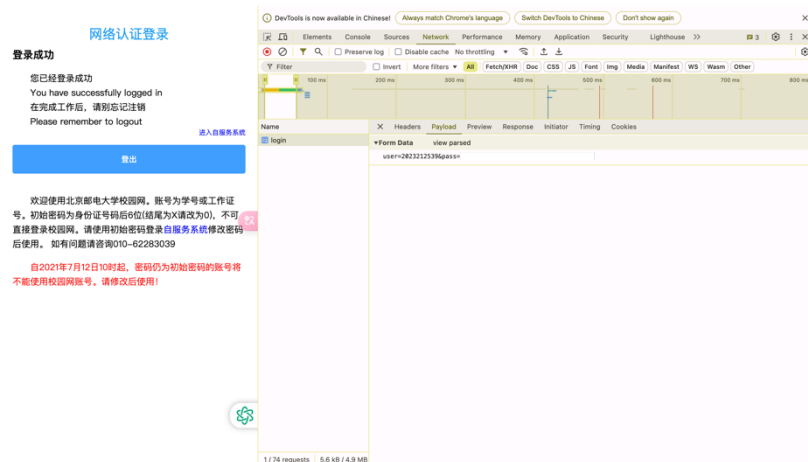
4 实验原理

完成本实验首先要解决的基本问题是：BUPT 校园网使用网页登录的方式认证，我们在上网时，需要在认证网页输入用户名+密码登录。

在网页登录时，注意到认证页面（<http://10.3.8.211/login>），如果尚未登录，输入用户名、密码，然后点击确定可以登录校园网；如果已经登录，会显示登陆成功的信息；如果已经登录成功，点击注销（<http://10.3.8.211/logout>），会注销成功，重新回到认证页面。

我们使用 chrome 浏览器自带的网页开发工具来找出为了登录，我们需要发送给校园网网关的数据和应答的数据。

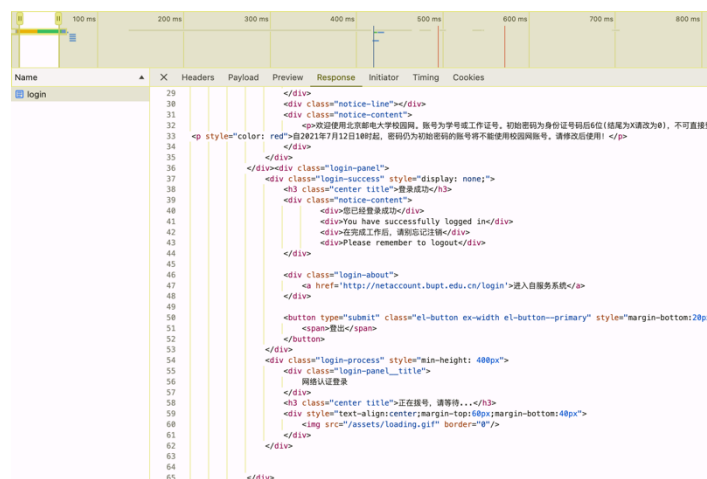
在认证页面（<http://10.3.8.211/index>）按下 F12，打开 chrome 开发者工具，选中开发者工具的 network 菜单-headers 选项卡：



并且调试工具显示：Request URL: <http://10.3.8.211/login> <http://10.3.8.211/login> 就是登录发送的网址。

在下部有 Form data 处点击 view source，显示格式是 user=XXX&pass=YYY。

如果输入正确的用户名+密码后，点击 response 菜单，可以看到登录成功时，



网关返回的网页的源文件。

通过以上操作，得出在登录时，可以用 `http` 的 `Post` 方法往网关 `http://10.3.8.211/login` 提交用户名（`user=用户名`）和密码（`pass=密码`），然后从返回的网页中匹配某个特征字符串，就可以得知是否登录成功。例如，登录成功时，会返回 “`You have successfully logged in`”，而失败则没有。

同理，通过用 `http` 的 `Get` 方法请求网关 `http://10.3.8.211/`，从返回的网页中匹配某个特征字符串（`successfully logged in`），就可以得知登录的状态。

注销时，需要 `http` 的 `Get` 方法请求 `http://10.3.8.211/logout`，然后浏览器会跳转到 `http://10.3.8.211/index` 页面，通过匹配此页面的特征字符串（如 “输入账号”），就可以得知注销的状态。

整个分析过程如下：



5 实验步骤

一、打开开发环境

- 在机房电脑打开 `C:\CxcyCourse\AppInventor2018PersonalEdition_Win\`;
- 启动 `AppInventor.cmd`，启动离线版的 `AppInventor2`;
- 打开 `chrome` 浏览器，输入 `localhost:8888`;

二、新建项目

- 点击 `Poejects->Start new project`;
- 创建项目，名称为 `test1`;

三、设计界面

- 进行界面设置，用到如下组件如下表:

组件类型	组件种类	命名	显示文本	作用
VerticalArrangement 垂直布局组件	Layout 布局组件	VerticalArrangement1	无	使整体界面呈垂直布局
TableArrangement 表格布局组件	Layout 布局组件	TableArrangement1	无	使 Button 和 Label 等横向居中布局
Label 标签组件	User Interface 用户界面组件	Label_username	用户名	显示用户名
Label 标签组件	User Interface 用户界面组件	Label_password	密码	显示密码
TextBox 文本框组件	User Interface 用户界面组件	TextBox_username	无	输入用户名
PasswordTextBox 文本框组件	User Interface 用户界面组件	PasswordTextBox_passwords	无	输入密码
Button 按钮组件	User Interface 用户界面组件	Button_login	登录	登录
Button 按钮组件	User Interface 用户界面组件	Button_logout	注销	注销
Button 按钮组件	User Interface 用户界面组件	Button_showStatus	显示状态	显示当前登录状态
Sound 音效播放器	Media 多媒体组件	Sound1		登录和注销时发出声音和震动
AccelerometerSensor 加速度传感器	Sensors 传感器组件	AccelerometerSensor1		检测手机晃动
Web 客户端	Connectivity 通信连接组件	Web1		发送 HTTP 请求并获取应答

● 添加 Layout 组件并设置属性：

- 拖入 VerticalArrangement 组件：用鼠标点中 Layout 下的 VerticalArrangement，拖动到右侧 Viewer 面板的 Screen1 中；
- 点击 Components 中的 VerticalArrangement1，最右侧的 Properties 将显示点中的 VerticalArrangement1 的属性，设置 Properties-》AlignHorizontal 和 AlignVertical 都为 center，表示在水平和垂直方向都按照中心对齐；
- 设置 Height 属性为 80 percent；
- 设置 Width 属性为 Fill parent；

属性面板

垂直布局1

水平对齐
居中 : 3 ▾

垂直对齐
居中 : 2 ▾

背景颜色
☐ 透明

高度
充满...

宽度
充满...

图片
无...

允许显示
☒

属性面板

表格布局1

列数
2

高度
自动...

宽度
自动...

行数
2

允许显示
☒

- 按照同样的方法，拖入 TableArrangement1 到 VerticalArrangement1 中，并设置属性；
- 添加 Button 和 TextBox 等可视组件并设置属性：
 - 拖入 2 个 Label 标签组件（用户名、密码）：用鼠标点中 User Interface 下的 Label，拖动到右侧 viewer 中 TableArrangement1 中相应的位置；
 - 拖入 1 个 TextBox 文本框组件：用鼠标点中 User Interface 下的 TextBox，拖动到右侧 viewer 中 TableArrangement1 中相应的位置；
 - 拖入 1 个 PasswordTextBox 文本框组件：用鼠标点中 User Interface 下的 PasswordTextBox，拖动到右侧 viewer 中 TableArrangement1 中相应的位置；
 - 拖入 3 个 Button 按钮组件（登录、注销、显示状态）：用鼠标点中 User Interface 下的 Button，拖动到右侧 viewer 中 TableArrangement1 中相应的位置；
 - 设置组件所显示的文本；
 - 重命名组件；



- 添加 Web 客户端等非可视组件并设置属性：
 - 拖入 1 个 Web（Web 客户端）框组件：用鼠标点中 Connectivity 通信连接下的 Web，拖动到右侧 Viewer 中。所有的非可视组件都显示在界面的下方。
 - 拖入 1 个 Notifier（对话框）组件：用鼠标点中 User Interface 用户界面下的 Notifier，拖动到右侧 Viewer 中。
 - 拖入 1 个 Sound（音效播放器）组件：用鼠标点中 Media 多媒体下的 Sound，拖动到右侧 Viewer 中。
 - 拖入 1 个 AccelerometerSensor（加速度传感器）组件：用鼠标点中 Sensor 传感器下的 AccelerometerSensor，拖动到右侧 Viewer 中。



- 登录成功与失败时手机将发出相应的声音提醒用户，首先需要上传声音的素材文件。
- 点击界面右下角 Media 面板，选择 Upload File，上传下列文件：
 - 音效文件：logon.wav、logoff.wav
 - 图片素材：Exp1_Logo.png、Exp_bg1.jpg



- 设置 Screen 属性：
 - 为 Screen1 修改全局属性，包括：应用名称、主题样式、App 图标等。



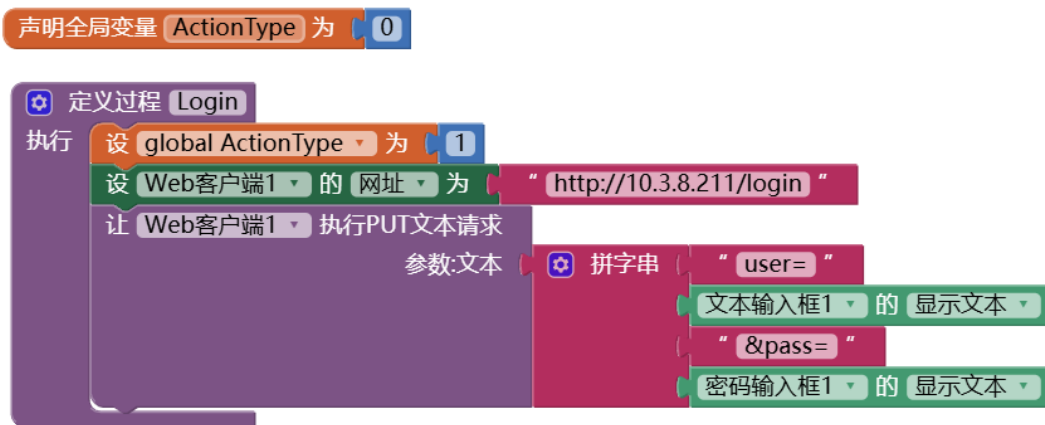
- 界面效果：



四、编写代码

- 处理登录 Button 点击事件：

- 先定义一个全局变量 `ActionType` 用于区分请求类型，“登录、注销、显示状态”3 个按钮事件都会使用 web 客户端向网关发送请求；
- 创建一个过程 `Login`：因为加速度传感器检测到晃动和点击“登录”按钮的操作都是发送登录请求，为防止代码冗余，将这 2 个相同的操作提取为一个过程 `Login`；
- 基本过程是：定义一个请求登录网关的函数 `Login`，将 `ActionType` 设置 1，将 web 客户端的 url 网址设置为 `http://10.3.8.211/login`，将 web 客户端的 `Post` 参数设置为之前分析所得的格式，即 ***user=用户名&pass=密码***；
- 在 `Button_login` 的 `Click` 事件中调用 `Login` 过程；



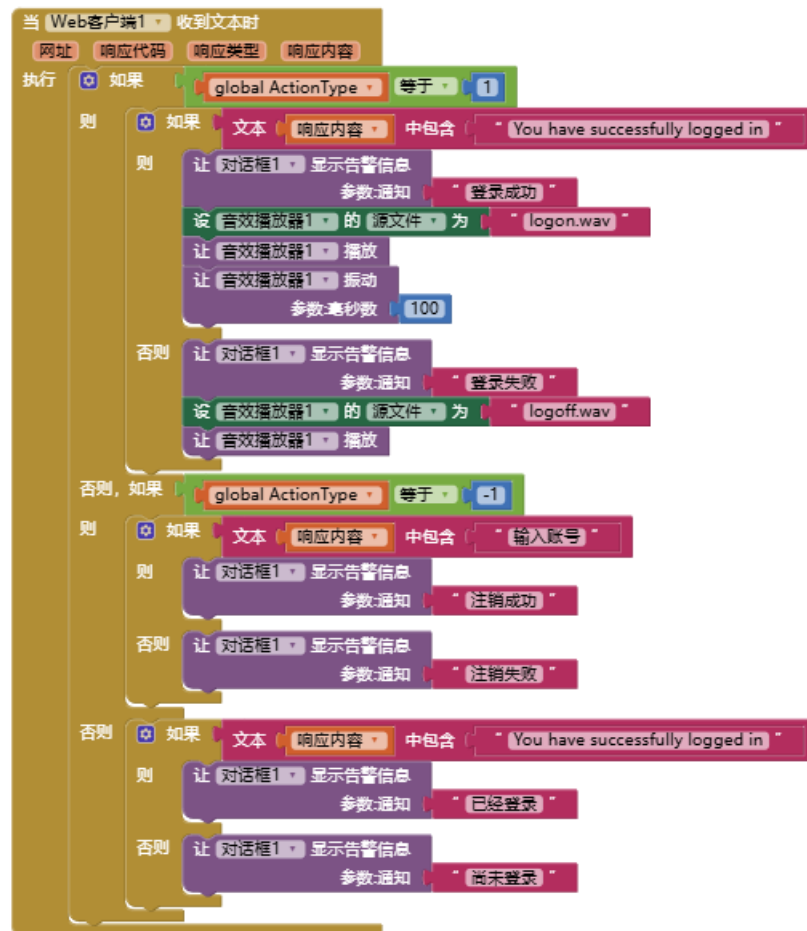
- 处理注销 Button 点击事件：注销操作是使用 web 客户端向 <http://10.3.8.211/logout> 发送 get 请求，无需输入账户；



- 处理显示状态 Button 点击事件：显示登录状态操作是使用 web 客户端向 <http://10.3.8.211> 发送 get 请求，无需输入账户。



- 处理 Web 客户端 Web1 的响应事件：“登录、注销、显示状态” 3 个按钮事件都会使用 web 客户端向网关发送请求，当网关返回应答数据时，web 客户端 Web1 的 GotText 事件将被调用。在此事件中根据请求的类型做响应的处理；



五、测试应用：

- 打开 C:\CxcyCourse\AI2Companion_Win7_64bit\ailaunch.bat，启动桌面版 AI 伴侣；
- 在开发环境中，选择 Connect-》AI Companion，通过编码进行连接；
- 点击“显示状态”按钮，显示登录状态。
- 输入用户名和密码，点击“登录”按钮，提示登录成功。点击“注销”按钮，显示注销成功。均正确。

六、处理加速度传感器晃动事件（由于无安卓手机，仅编写代码，并未测试）

- 加速度传感器 AccelerometerSensor1 晃动时，产生 Shaking 事件，在此事件中调用 Login 过程，则可以触发登录操作。



6 实验结果与分析



初始界面，提示词等正确显示。

输入账号密码，显示登录成功，且密码为隐藏显示（相较于基本要求增加的功能）。

点击显示状态，能正确显示当前登录状态；

再点击注销按钮，成功注销登录。



7 实验总结

除了按照实验手册完成基本实验内容，还可以通过以下方面对实验进行优化，如：

1. 为了解决按钮位置的问题，可以调整界面的布局，使按钮更符合用户手指操作习惯：使用底部固定栏：将常用按钮（如登录、注销等）放置在屏幕底部的固定栏中。
2. 合并登录和注销按钮：动态显示按钮状态：根据用户是否已登录，动态更新按钮的文本和功能。在未登录状态显示“登录”，点击后执行登录操作；已登录状态显示“注销”，点击后执行注销操作。还可以设置不同状态相应的颜色作区分。

实验中主要遇到的问题如下：第一次测试过程中，输入正确的帐号密码不能正确登录，检查代码和输入均无误，再次检查网页端 POST 请求发送的信息，发现是由于密码中使用了特殊符号，网页端做了**转义操作**，导致在 APP 测试段输入原符号，不能正确响应。在输入转义后的字符串后，成功登录。

实验二 扬翅的小鸟

1 实验目的

1. 熟悉 App Inventor 2 开发环境和开发过程；
2. 理解事件处理程序在 App Inventor 2 开发中的重要作用
3. 熟悉 App Inventor 2 的组件布局、画布、定时器、音效等基本功能的使用
4. 熟悉列表等基本数据结构的使用
5. 熟悉判断等逻辑结构的使用
6. 理解精灵动画的实现原理

2 实验环境

1. 硬件环境
 - PC 微机（由于在机房电脑上编写和调试，部分实验截图可能不全）
2. 软件环境
 - Windows 操作系统
 - App Inventor2018 离线版（AppInventor2018PersonalEdition_Win）桌面版 AI 伴侣（AI2Companion_Win7_64bit）
3. Android 手机和安装在手机上的 MIT AI2 Companion App。

3 实验要求

本实验将要实现以下功能：

1. 在屏幕上创建游戏的背景。
2. 并在屏幕中间创建一只扇动翅膀的小鸟。
3. 同时小鸟发出扇动翅膀的声音。

实验要点有：

1. 打开 App Inventor 开发环境；

2. 使用组件设计器创建应用的外观;
3. 使用块编辑器为组件设定行为;
4. 使用组件设计器选择组件;
5. 从本地计算机加载媒体文件（声音和图像），并添加到应用中;
6. 用块编辑器来组装程序块，以此来设定组件行为;
7. 使用定时器实现动画效果;
8. 用 App Inventor 的实时测试功能对应用进行测试。

4 实验原理

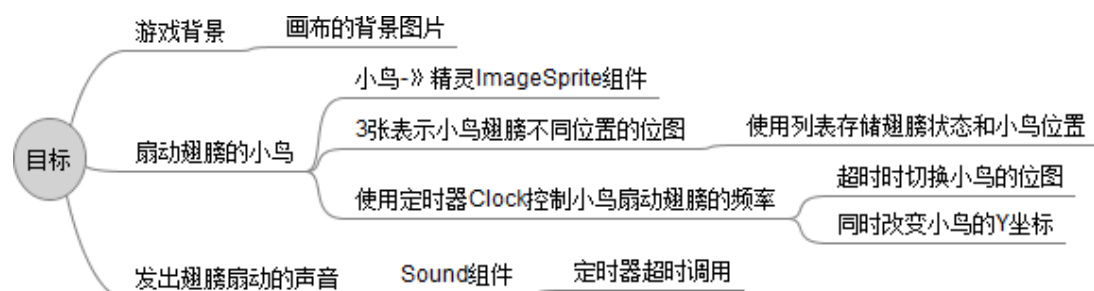
本实验使用的动画技术可称为精灵动画（Sprite Animation），它是一种电子版的序列帧动画，是以快速连续播放不同的静止画面，来产生角色的动态效果，就像电影胶片一般，通过快速切换来产生动画效果。

所谓精灵，就是一张张大小相同的位图。通常使用带有透明通道的图片格式保存，每一个区域都代表这个精灵在某一静止时刻的动作体现。

获取精灵动画的位图资源之后，在程序中只需将资源按单个动作图像大小进行切割，并存于数组中，在程序循环中根据时间的改变，顺序的循环显示不同的精灵动作，就可以实现出角色的动画效果。

对于本实验，可以使用 3 张位图表示小鸟翅膀的上中下 3 个位置，再配合以位置的上下，就可以模拟出扇动翅膀的动画效果。

我们将整个任务分解如下：



所以，实验的设计思路是，使用一个列表存储小鸟在上中下中 4 个状态的位图与位置，然后定时器超时时轮流切换这 4 个状态，这样每 4 个超时会完成了翅膀的一次扇动。

5 实验步骤

一、打开开发环境

- 在机房电脑打开 C:\CxcyCourse\AppInventor2018PersonalEdition_Win\;
- 启动 AppInventor.cmd，启动离线版的 AppInventor2;
- 打开 chrome 浏览器，输入 localhost:8888;

二、新建项目

- 点击 Poejects->Start new project;
- 创建项目，名称为 test2;

三、设计界面

- 上传以下素材：
 - 音效素材：wing.ogg 翅膀扇动的音效;
 - 图 像 素 材： background-night.png 游 戏 背 景、 base.png 地 基、 yellowbird-midflap.png 小 鸟 在 中 间 位 置 的 位 图、 yellowbird-upflap.png 小 鸟 向 上 扑 腾 的 位 图、 yellowbird-downflap.png 小 鸟 往 下 飞 的 位 图;



- 添加画布（Canvas），并设置画布属性如下：

属性面板

画布1

背景颜色
 默认

背景图片
background-night.png...

字号
14.0

高度
充满...

宽度
充满...

画笔线宽
1

画笔颜色
 默认

文本对齐
居中: 1 ▾

允许显示
☒

- 在 Canvas1 中添加地基精灵 (ImageSprite)，重命名为 ImageSprite_base，并设置属性如下：

属性面板

精灵1

启用
☒

方向
0

高度
自动...

宽度
自动...

间隔
100

图片
base.png...

旋转
☒

速度
0.0

允许显示
☒

X坐标
0

Y坐标
320

Z坐标

- 在 Canvas1 中添加小鸟精灵 (ImageSprite)，重命名为 ImageSprite_bird，并设置属性如下：

属性面板

精灵2

启用 ☒

方向

高度

宽度

间隔

图片

旋转 ☒

速度

允许显示 ☒

X坐标

Y坐标

Z坐标

- 添加一个定时器 (Clock)，并设置属性如下：

属性面板

计时器1

一直计时 ☒

启用计时 ☐

计时间隔

- 添加一个音效播放器 (Sound)，并设置属性如下：

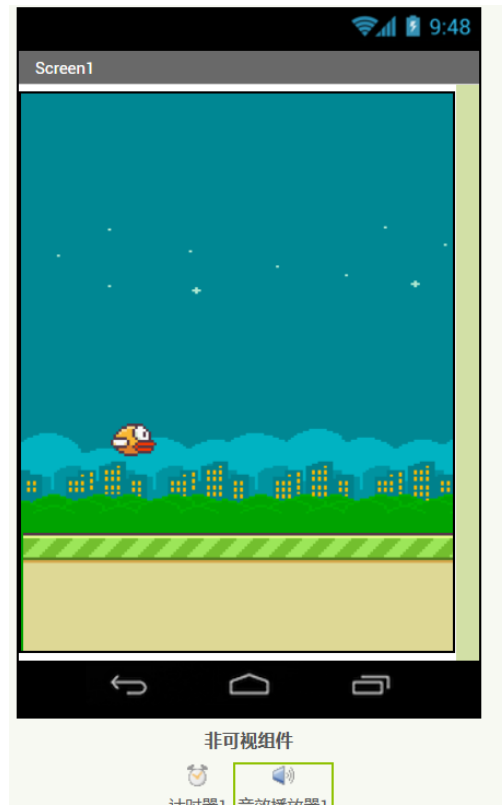
属性面板

音效播放器1

最小间隔(毫秒)

源文件

- 最终界面如下：

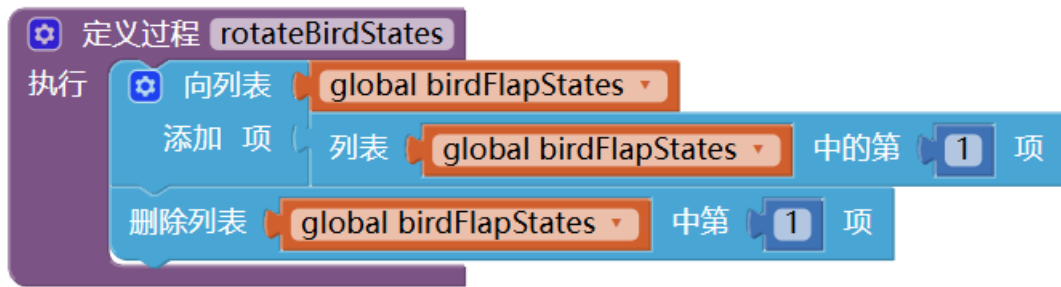


四、编写代码

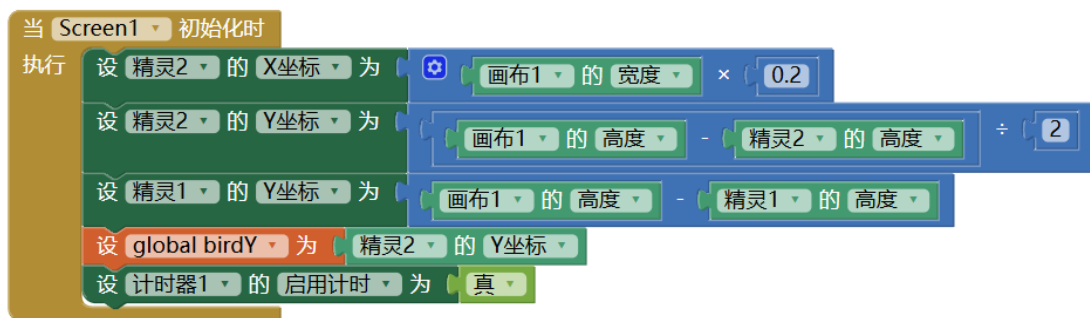
- 创建一个全局列表 `birdFlapStates`，该列表的每一项又是一个列表，分别存储小鸟的位图与 Y 坐标偏移。



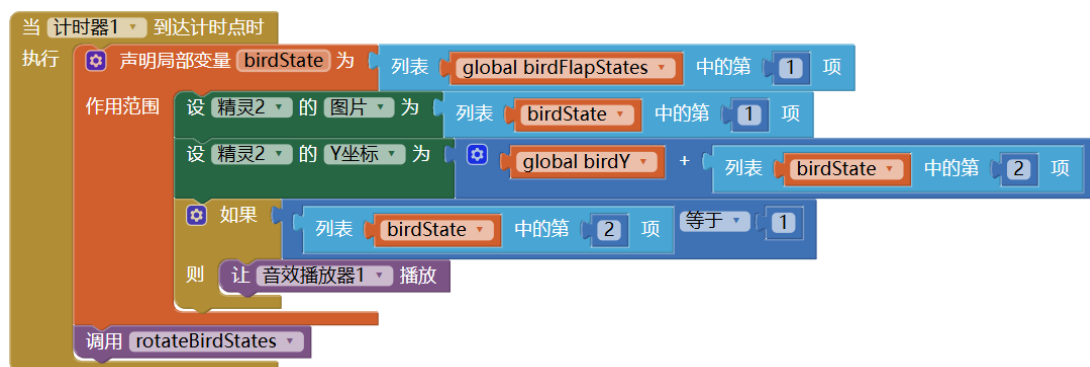
- 添加一个轮换列表状态的函数 `rotateBirdStates`。该函数的原理是把列表的第一个元素添加到列表末尾，然后删除第一个元素。以后每次都取列表的第一个元素，这就模拟了一次切换。



- 添加屏幕初始化事件的响应代码，完成完成界面元素的坐标设置，将 Canvas1 中的 base 和 bird 根据 Canvas1 的长宽进行调整等初始化。



- 添加一个表示小鸟初始 Y 位置的全局变量，并初始化为小鸟精灵的 Y 坐标，以后动画将以此为基准进行偏移。（见上）
- 添加定时器超时事件的响应代码，完成小鸟扇动翅膀的动画与声音。定时器超时，从 birdFlapStates 列表取出第一个元素，取出该元素中的小鸟位图与 Y 坐标偏移，设置到 ImageSprite_bird 的相应属性中。



- 调用 rotateBirdStates 完成列表中小鸟状态的切换。
- 通过滑动条改变定时器切换时间间隔（拓展功能）。

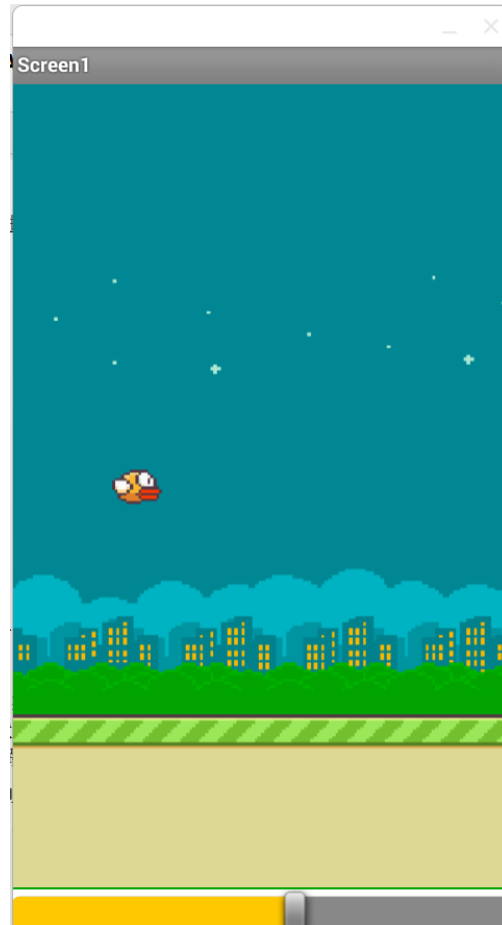
五、测试应用

- 打开 C:\CxcyCourse\AI2Companion_Win7_64bit\ailaunch.bat，启动桌面版 AI 伴侣；

- 在开发环境中，选择 Connect-》AI Companion，通过编码进行连接；
- 正确运行；

6 实验结果与分析

启动 AI 伴侣，拖动滑动条，观察到小鸟扇动翅膀频率随之正确改变：



7 实验总结

除了按照实验手册完成基本实验内容，还可以通过以下方面对实验进行优化，如：

1. 使用 4 个状态能实现动画的更平滑循环。使用 4 个状态可以更细化地表示翅膀的不同角度，增强视觉效果。如果使用 3 个状态，动画会显得不够流畅，尤其是在连续循环中，可能会出现不自然的跳跃；
2. 用 if...else 实现动画逻辑：如果不用列表而用 if...else 判断，可以通过变

量保存当前状态，然后根据状态进行逻辑判断，不过缺点是代码比较冗长；

3. 允许坐标超出范围：可以通过逻辑判断决定是否将小鸟的位置限制在画布内，或者超出后触发特殊逻辑（如从另一侧重新进入画布）；
4. 定时器间隔不能太小，如果太小，动画效果可能不与设置完全一致，比如屏幕刷新率（通常为 60Hz）会限制动画帧率，或动画逻辑较复杂，每帧的计算时间较长，实际间隔会比预设值大；
5. 在界面底部添加滑动条调整小鸟振翅频率（即定时器间隔）；

实验三 下坠的小鸟

1 实验目的

1. 掌握 App Inventor 2 开发过程
2. 了解 App Inventor 2 调试程序的方法
3. 熟悉使用定时器实现和控制动画的方法
4. 检测并处理用户输入的点击事件
5. 碰撞检测
6. 多屏程序的设计
7. 使用过程（procedure）等技术组织稍微复杂的逻辑

2 实验环境

1. 硬件环境
 - PC 微机（由于在机房电脑上编写和调试，部分实验截图可能不全）
2. 软件环境
 - Windows 操作系统
 - App Inventor2018 离线版（AppInventor2018PersonalEdition_Win）桌面版 AI 伴侣（AI2Companion_Win7_64bit）
3. Android 手机和安装在手机上的 MIT AI2 Companion App。

3 实验要求

本实验将要在实验 1 的基础上增加以下功能：

1. 增加启动游戏的图像，当用户点击时，切换到游戏交互界面。
2. 在游戏交互界面，小鸟将以一定的加速度做自由落体运动。
3. 用户点击屏幕后，小鸟将获得一个向上的恒定速度。
4. 小鸟在上升下坠过程中，需要模拟抬头和低头动作。

5. 小鸟在碰撞到地基后，停止游戏并显示游戏结束消息和碰撞音效。

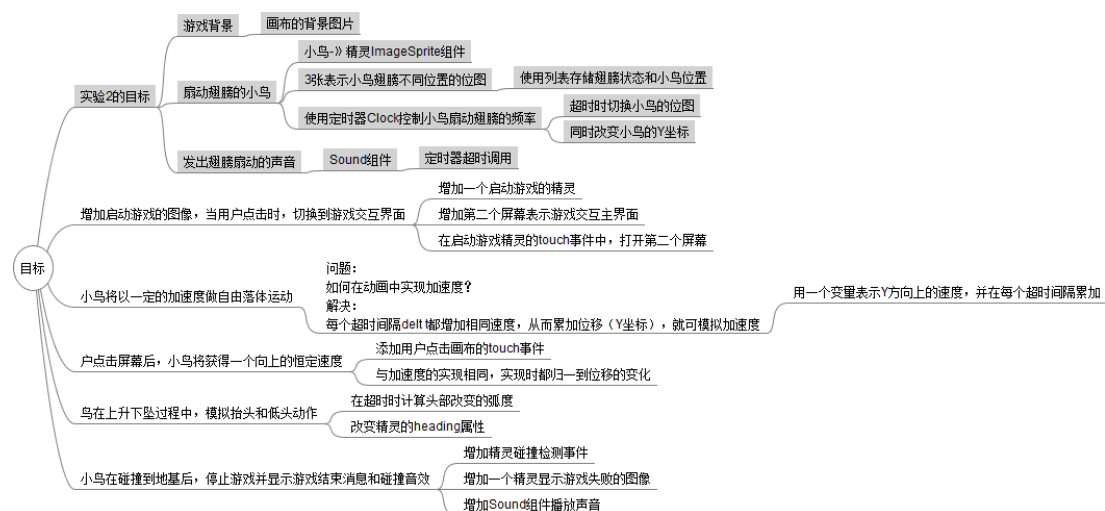
实验要点有：

1. 使用组件设计器添加多个屏幕，设计相应屏幕的外观；
2. 使用全局变量记录小鸟的状态，并根据用户输入修改小鸟状态；
3. 使用定时器实现小鸟扇动翅膀和抬头低头的动画效果；
4. 使用程序控制屏幕元素的布局，在屏幕之间传递参数；
5. 使用程序模拟物理世界行为的方法；
6. 响应精灵的碰撞检测事件。

4 实验原理

本实验在上一个实验的基础上增加功能。主要是小鸟除了扇动翅膀外，将获得一个向下的恒定加速度，需要用户点击屏幕，给予一个向上的初始速度，才能不因为碰到地基而导致游戏失败。需要注意的是 Y 方向上加速度与初始速度对小鸟运动的影响。所以需要增加一个全局变量记录小鸟的速度，因为超时间隔是一定的，所以位移的变化量就是速度的变化量，这样就可以用速度直接计算在每个超时间隔的 Y 方向位移值。

我们可以将实验设计思路拆分如下：



5 实验步骤

一、打开开发环境

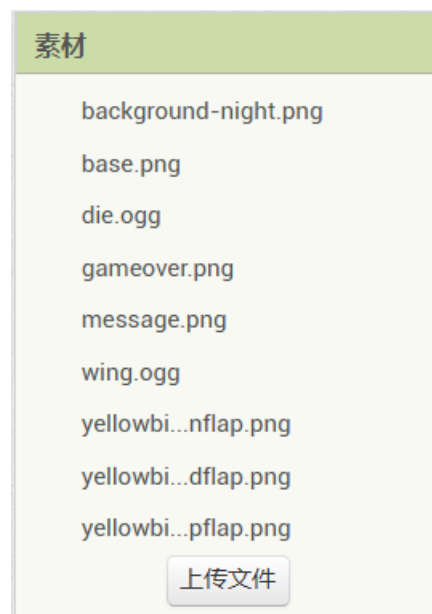
- 在机房电脑打开 C:\CxcyCourse\AppInventor2018PersonalEdition_Win\;
- 启动 AppInventor.cmd, 启动离线版的 AppInventor2;
- 打开 chrome 浏览器, 输入 localhost:8888;

二、新建项目

- 点击 Poejects->Start new project;
- 创建项目, 名称为 test3;

三、设计界面

- 上传以下素材:
 - 音效素材: die.ogg 小鸟死亡音效;
 - 图像素材: message.png 启动游戏的图片、gameover.png 显示游戏结束的图片;



- 在 Canvas1 中添加精灵 (ImageSprite), 重命名为 ImageSprite_message, 并设置属性如下:

属性面板

精灵3

启用

☒

方向

0

高度

自动...

宽度

自动...

间隔

100

图片

message.png...

旋转

☒

速度

0.0

允许显示

☒

X坐标

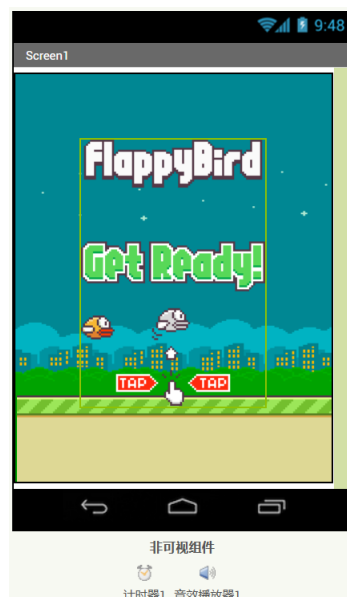
64

Y坐标

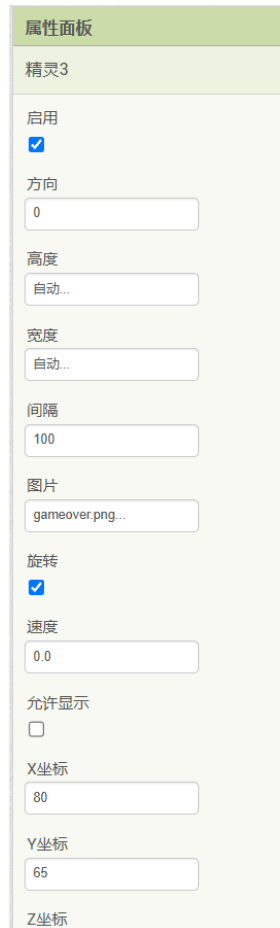
64

Z坐标

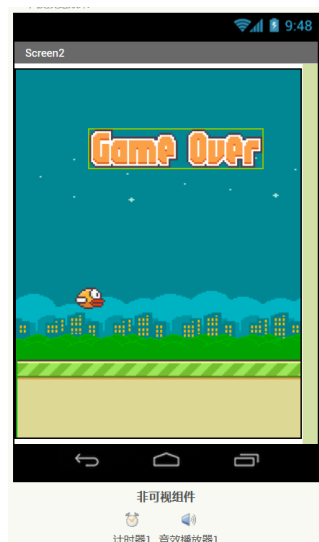
- Screen1 界面如下：



- 创建 Screen2，按照上一个实验增加以下组件并设置属性；
- 在 Canvas1 中添加精灵（ImageSprite），重命名为 ImageSprite_over，并设置属性如下：

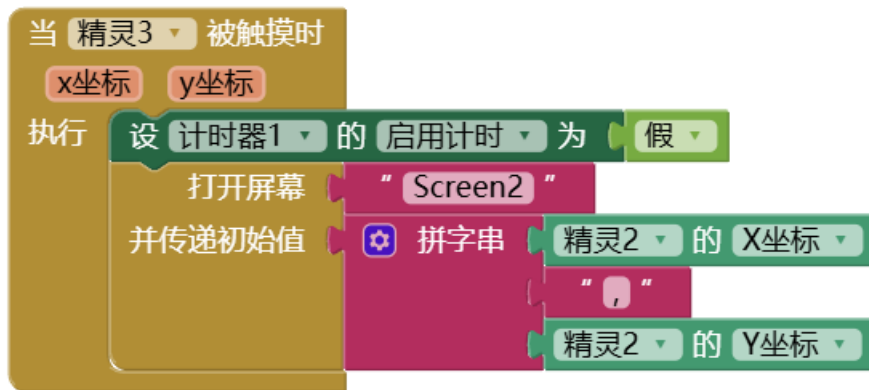


- Screen2 界面如下：

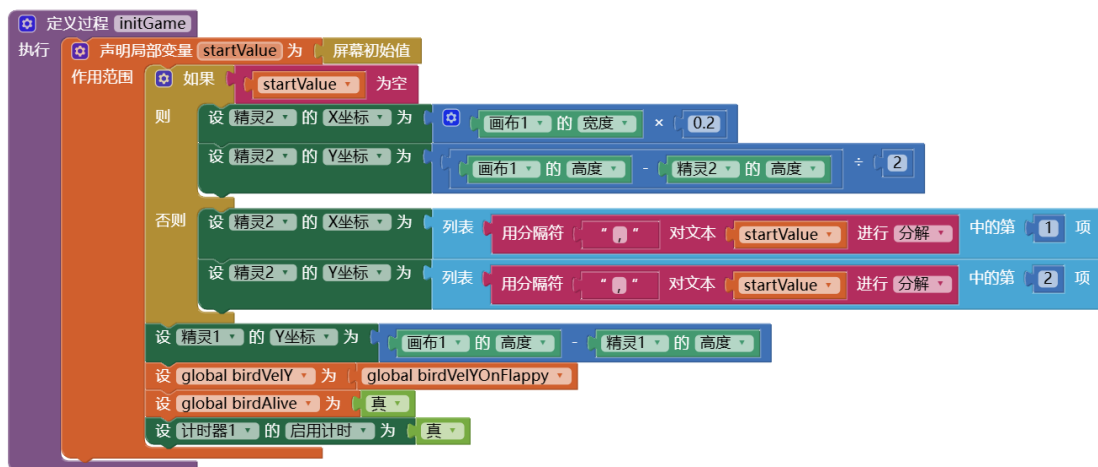


四、编写代码

- 处理 Screen1 的 touched 事件，打开第二个屏幕，并将小鸟坐标传给 Screen 2；



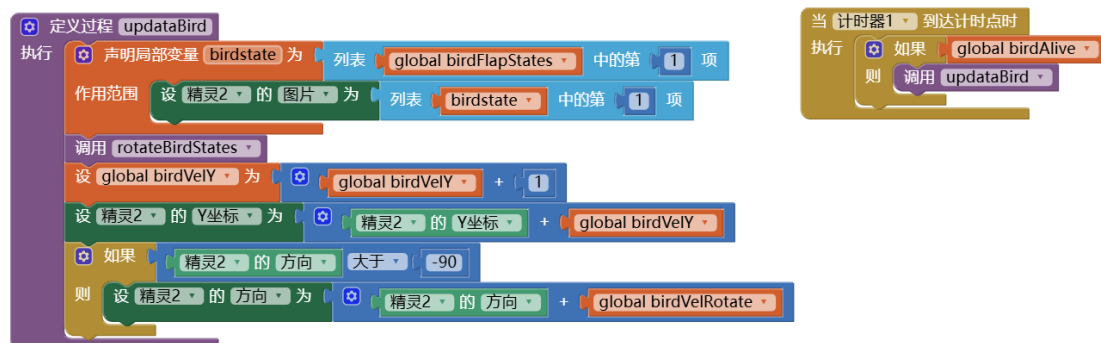
- 在 Screen2 中，创建一个全局列表 birdFlapStates，以及轮换列表状态的函数 rotateBirdStates；（与实验二相同）
- 添加 Screen2 初始化事件的响应代码，完成界面元素的坐标设置以及相关全局变量和游戏组件的初始化，启动定时器，进入定时器循环；



- 添加一个表示小鸟 Y 方向上初始速度的全局变量 birdVelY，并初始化为一个合适的值，以后小鸟的速度将以此为初值进行计算。
- 添加一个表示小鸟生命状态的全局变量 birdAlive，用于表示游戏是否结束，控制点击屏幕等事件是否需要响应。



- 添加定时器超时事件的响应代码，完成小鸟扇动翅膀、改变位移、改变抬头角度的动画。



- 添加屏幕点击事件的响应代码，完成赋予小鸟初始速度、抬头、发出翅膀声音的效果。



- 添加碰撞检测事件的响应代码，显示游戏结束、改变小鸟位置与朝向、停止定时器、播放碰撞声音。



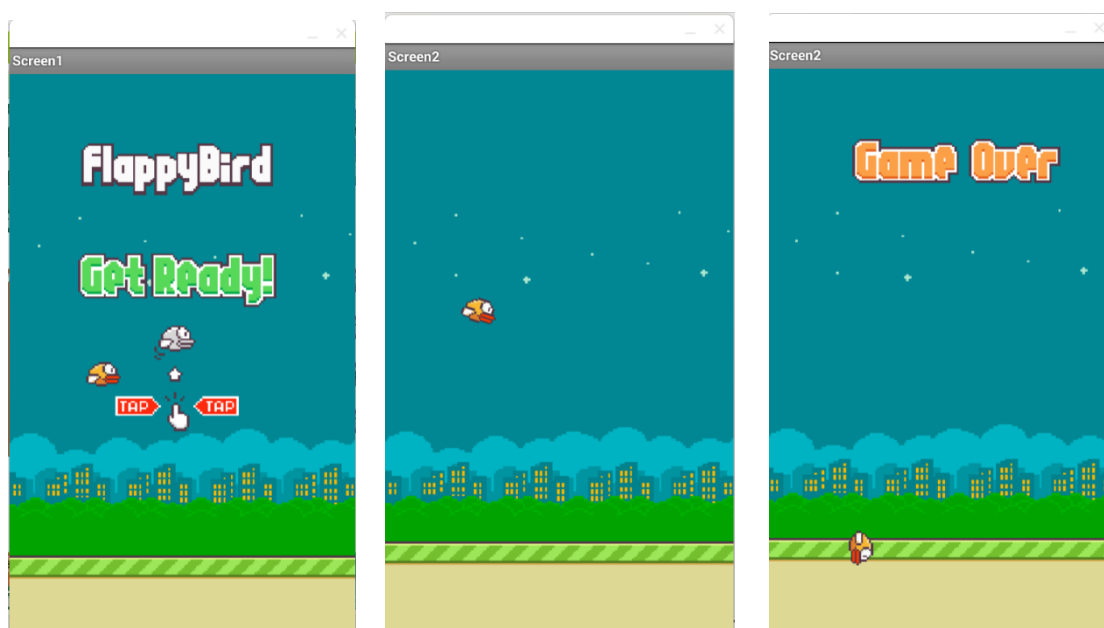
五、测试应用

- 打开 C:\CxcyCourse\AI2Companion_Win7_64bit\ailaunch.bat，启动桌面版 AI 伴侣；

- 在开发环境中，选择 Connect-》AI Companion，通过编码进行连接；
- 正确运行，小鸟坠落至地面，出现 Game Over 图标；

6 实验结果与分析

进入游戏，先为初始界面，点击屏幕后，游戏正常开始，点击屏幕，小鸟能向上飞，否则则下坠，当与地面碰撞时，正确死亡，并出现 Game Over 图标。



7 实验总结

除了按照实验手册完成基本实验内容，还可以通过以下方面对实验进行优化，如：

1. 可以使用一个屏幕来实现游戏初始界面和游戏正式交互界面的功能，在正式开始游戏后，将初始界面的图标的 `enable` 属性设置为 `false`，其余初始化操作不变即可；
2. 要让小鸟、地基等精灵在不同手机屏幕上位置保持一致，可以使用相对布局，获取屏幕尺寸，将坐标定义为屏幕宽高的百分比，而非固定值，在绘制元素时根据屏幕尺寸计算实际坐标。
3. 初始速度与下落加速度对效果的影响：初始速度越大，小鸟飞得越快，玩家会感到更难控制，初始速度过低可能导致游戏节奏变慢，不够有趣；下落加速度大时，小鸟下落速度会更快，玩家需要更频繁地点击以保持小鸟飞行，加速度过小时，下落会显得不真实，失去物理感；

实验四 避障的小鸟

1 实验目的

1. 掌握使用画布、精灵、定时器实现动画的方法
2. 碰撞检测与边界检测
3. 随机数的使用
4. 使用过程（procedure）等技术组织复杂的逻辑

2 实验环境

1. 硬件环境
 - PC 微机（由于在机房电脑上编写和调试，部分实验截图可能不全）
2. 软件环境
 - Windows 操作系统
 - App Inventor2018 离线版（AppInventor2018PersonalEdition_Win）桌面版 AI 伴侣（AI2Companion_Win7_64bit）
3. Android 手机和安装在手机上的 MIT AI2 Companion App。

3 实验要求

本实验将需要在实验 2 基础上增加以下功能：

1. 在屏幕 2 中随机生成上下 2 根柱子
2. 柱子以恒定的速率从右往左运动
3. 用户通过点击控制小鸟的速度从柱子之间穿过
4. 穿过中如果碰到柱子则游戏失败，停止游戏并显示游戏结束消息和碰撞音效。

实验要点有：

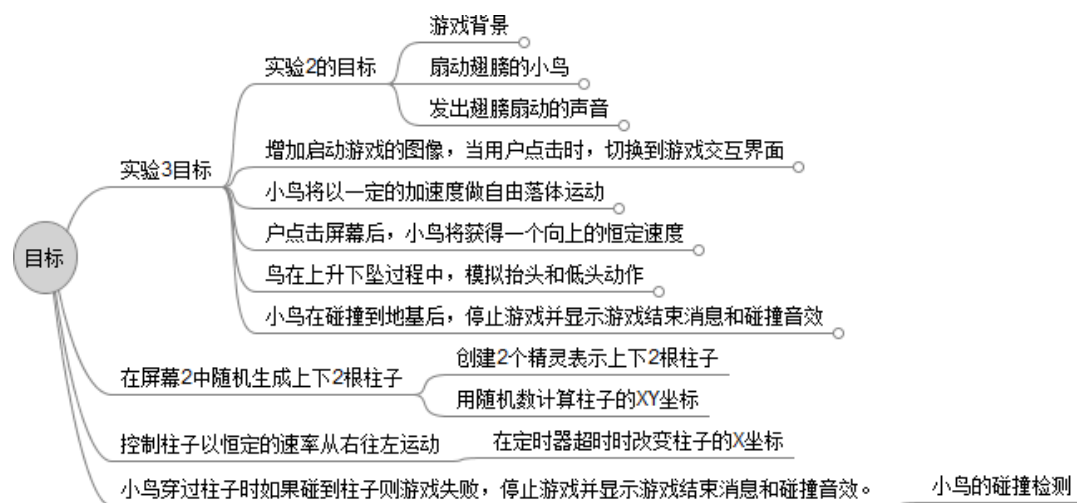
1. 使用随机数控制 2 根柱子的起始位置
2. 以柱子的从右往左移动模拟小鸟的从左往右飞

3. 使用全局变量记录游戏的状态，包括小鸟柱子等
4. 响应精灵的碰撞检测事件。

4 实验原理

本实验在上一个实验的基础上增加功能。主要是需要随机生成 2 根柱子，并控制小鸟从柱子穿过。采用的办法是让柱子从右往左运动，模拟小鸟的穿柱子操作。

我们可以将实验设计思路拆分如下：



5 实验步骤

一、打开开发环境

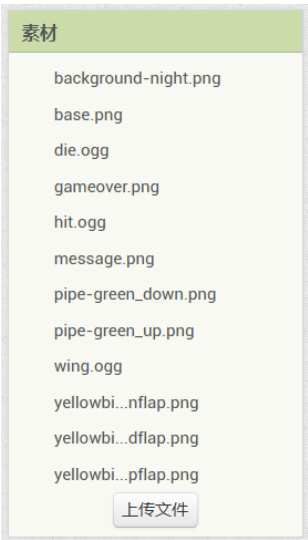
- 在机房电脑打开 C:\CxcyCourse\AppInventor2018PersonalEdition_Win\;
- 启动 AppInventor.cmd，启动离线版的 AppInventor2;
- 打开 chrome 浏览器，输入 localhost:8888;

二、新建项目

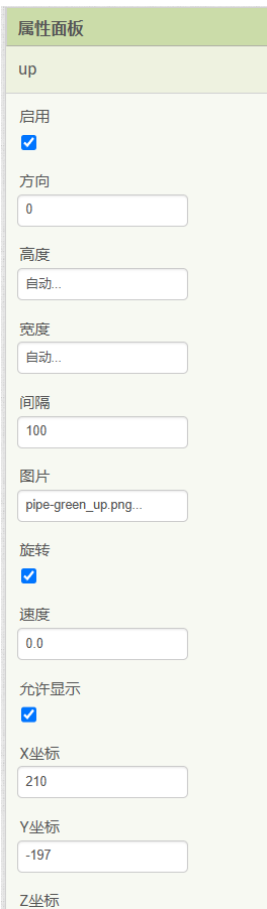
- 点击 Poejects->Start new project;
- 创建项目，名称为 test4;

三、设计界面

- 上传以下素材：
 - 图像素材： pipe-green_down.png 和 pipe-green_up.png 柱子的图像素材；

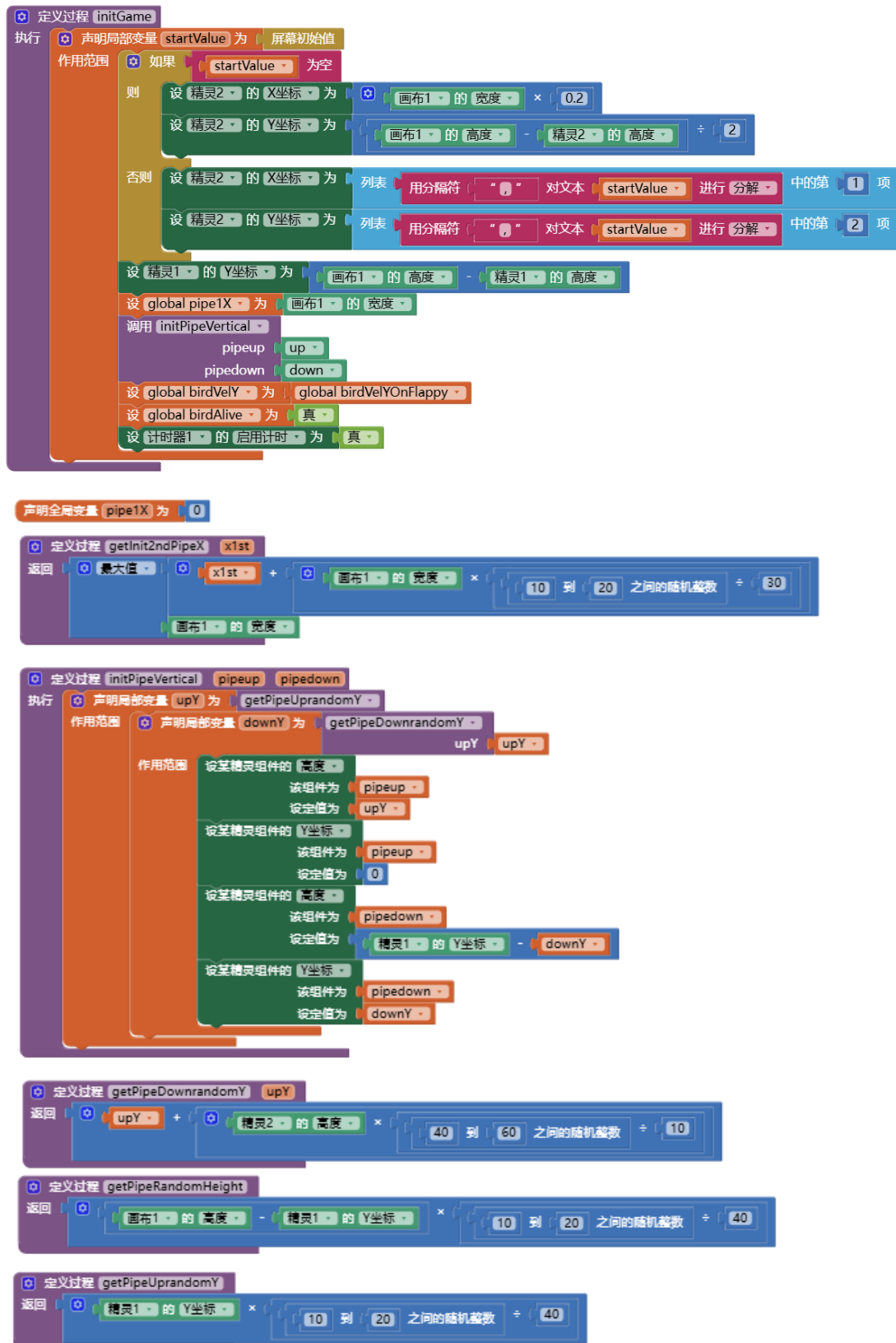


- 在 Canvas1 中添加2个精灵(ImageSprite_pipe1up 和 ImageSprite_pipe1down Screen1) 表示柱子，设置图像、是否显示等属性如下：



四、编写代码

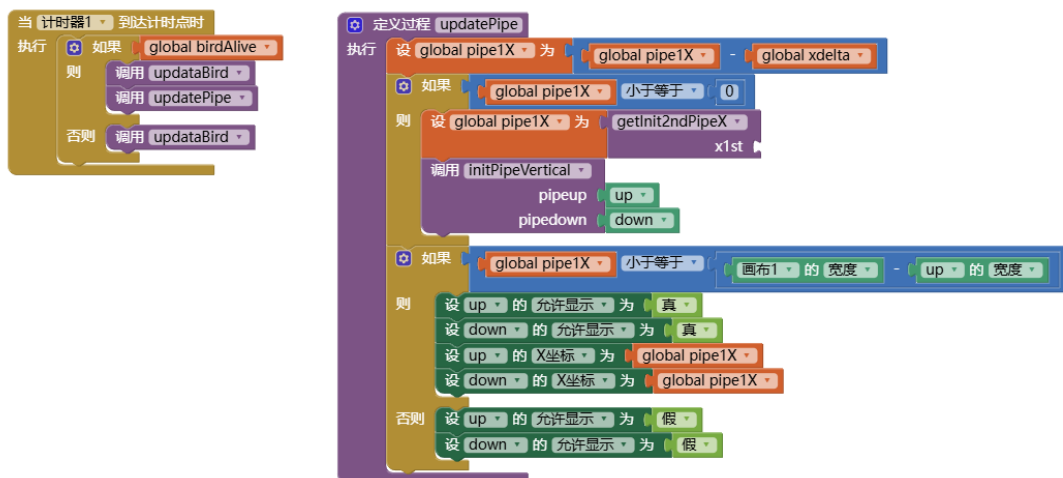
- 修改屏幕 2 的初始化函数，计算随机生成的上下 2 根柱子的 XY 坐标和高度；



- 在 Screen2 中，创建一个全局列表 birdFlapStates，以及轮换列表状态的函数

rotateBirdStates;（与实验二相同）

- 在定时器的超时事件处理函数中增加柱子从右往左移动的功能。每次超时时，柱子的 x 坐标减去固定的数值模拟匀速运动；



- 在精灵的碰撞检测函数中增加柱子与小鸟碰撞的判断代码，如果发生碰撞，停止游戏，显示游戏结束信息；

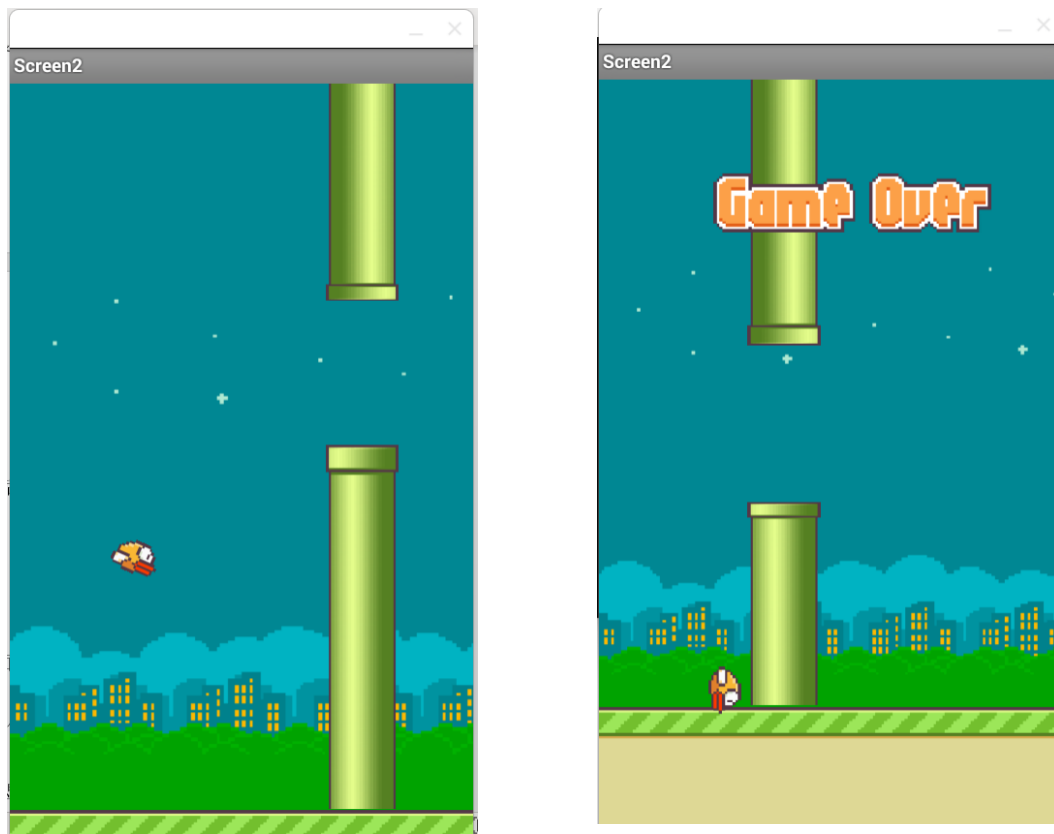


五、测试应用

- 打开 C:\CxcyCourse\AI2Companion_Win7_64bit\ailaunch.bat，启动桌面版 AI 伴侣；
- 在开发环境中，选择 Connect-》AI Companion，通过编码进行连接；
- 正确运行；

6 实验结果与分析

初始界面与实验三相同，不再展示，下图是正确开始游戏后，



可见，柱子正确生成，且不会发生重叠。小鸟与柱子发送碰撞触发死亡。

7 实验总结

除了按照实验手册完成基本实验内容，还可以通过以下方面对实验进行优化，如：

1. 确定随机数范围以满足不超出画布且不与小鸟相碰或在小鸟左边：
X 坐标：柱子必须在小鸟的右侧；
Y 坐标：柱子的 Y 坐标范围应使其完全在画布范围内显示。
不与小鸟相碰：柱子的 X 坐标初始值与小鸟的 X 坐标要保持一定距离，以避免柱子生成时立即碰撞。
2. 为了让柱子高度超出屏幕范围但仍正确显示，需要调整柱子绘制的起始位置，使其看起来部分隐藏在屏幕外，再根据屏幕高度计算绘制区域。
3. 小鸟成功通过柱子的条件是：

小鸟的 X 坐标 > 柱子的右边缘 X 坐标。

且小鸟在屏幕内时没有与柱子发生碰撞。

4. 增加计分功能：定义全局变量 `score` 记录当前分数，当小鸟通过柱子时，增加分数，游戏结束时显示分数；（第三和第四点在实验五中说明）

实验五 飞扬的小鸟

1 实验目的

1. 综合实验 1、2、3 中使用过各种编程元素：的画布、精灵、定时器、音效等组件，判断、循环等程序控制手段，列表、全局变量、随机数等数据手段，初始化、定时器、碰撞检测等事件处理，实现一个较为完整的游戏。
2. 使用变量、过程、和事件处理程序组织复杂的逻辑。
3. 熟悉程序状态的控制。
4. 了解如何保存数据到本地数据库或从本地数据库中读取数据。

2 实验环境

1. 硬件环境
 - PC 微机（由于在机房电脑上编写和调试，部分实验截图可能不全）
2. 软件环境
 - Windows 操作系统
 - App Inventor2018 离线版（AppInventor2018PersonalEdition_Win）桌面版 AI 伴侣（AI2Companion_Win7_64bit）
3. Android 手机和安装在手机上的 MIT AI2 Companion App。

3 实验要求

本实验将要在实验 3 基础上增加以下功能：

1. 在屏幕 2 中随机生成上下 2 根柱子，当柱子将到屏幕中央时，再随机生成上下 3 根柱子，之后屏幕上一共有 2 组上下 2 根共 4 根柱子。到柱子碰到屏幕左侧（柱子“消失”）时，在屏幕右侧再次随机生成 2 根柱子，如此循环不止。
2. 柱子的 XY 位置、高度和间隔都可以设计为随机的，这样柱子出现的时机、位置和彼此间的间隔都带有随机性。或者只将柱子的 Y 坐标设置成随机的，这

样程序更容易控制。

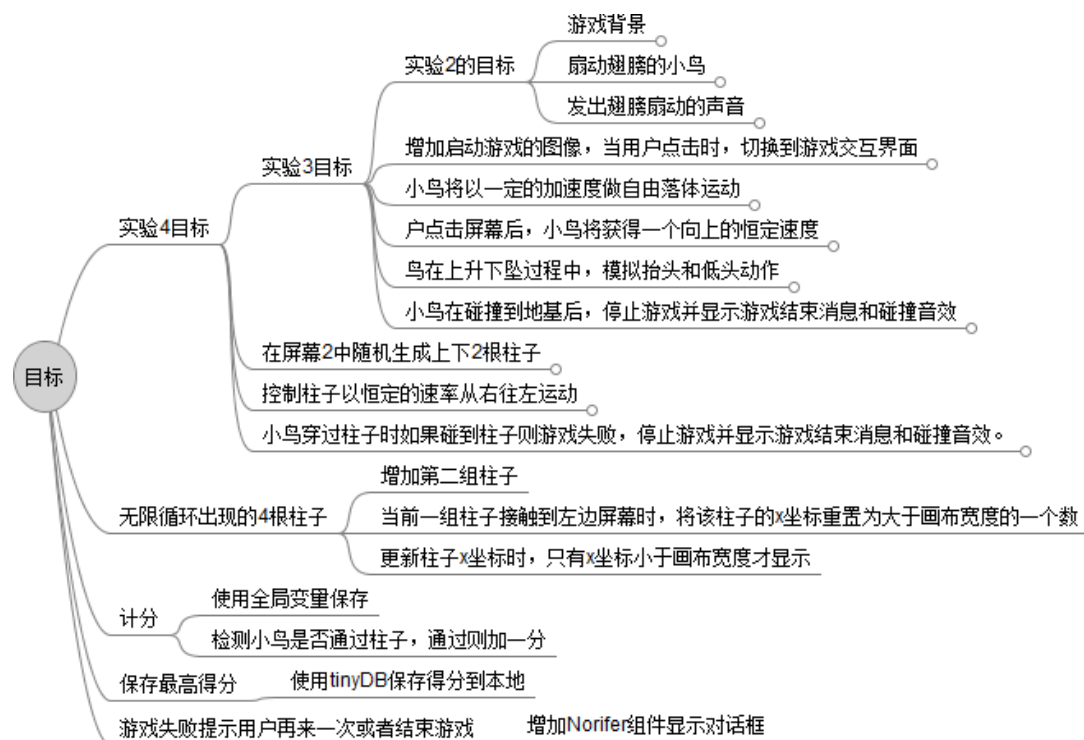
3. 增加计分功能，每次小鸟穿过一组柱子，就得一分。
4. 增加保存最高得分的功能，游戏结束时将最高得分从本地数据库读入，并与当前得分比对，将较大的结果存入本地数据库。
5. 穿过中如果碰到柱子则游戏失败，播放碰撞音效，小鸟继续自由下落，直到摔落到地基上，播放失败音效，停止游戏并显示游戏结束消息，结束消息中包括本地得分与最高得分，并提示用户再来一盘或者结束游戏。

实验要点有：

1. 使用随机数、列表和全局变量通过 4 根柱子模拟无限柱子的功能
2. 小鸟位置、初始速度、加速度、柱子的随机位置、高度、间隔等对游戏难度的影响。
3. 游戏状态的控制。
4. 理解精灵动画游戏通过静态帧循环的方式模拟动画的原理。
5. 记录与保存得分与最高得分。

4 实验原理

我们可以将实验设计思路拆分如下：



5 实验步骤

一、打开开发环境

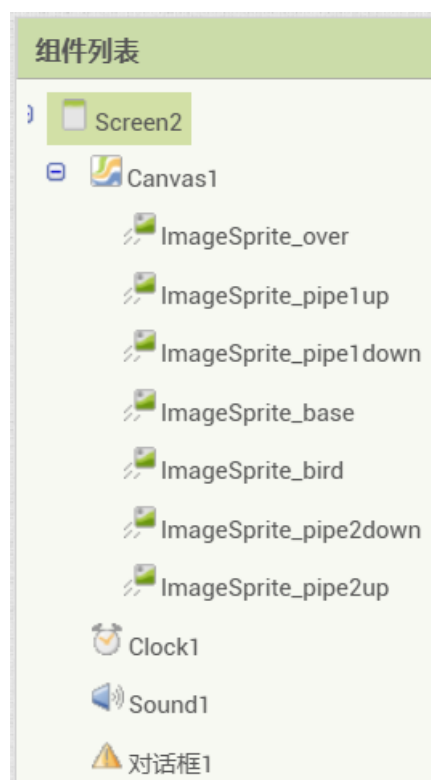
- 在机房电脑打开 C:\CxcyCourse\AppInventor2018PersonalEdition_Win\;
- 启动 AppInventor.cmd，启动离线版的 AppInventor2;
- 打开 chrome 浏览器，输入 localhost:8888;

二、新建项目

- 点击 Poejects->Start new project;
- 复制上一个实验的项目，重命名为 test4;

三、设计界面

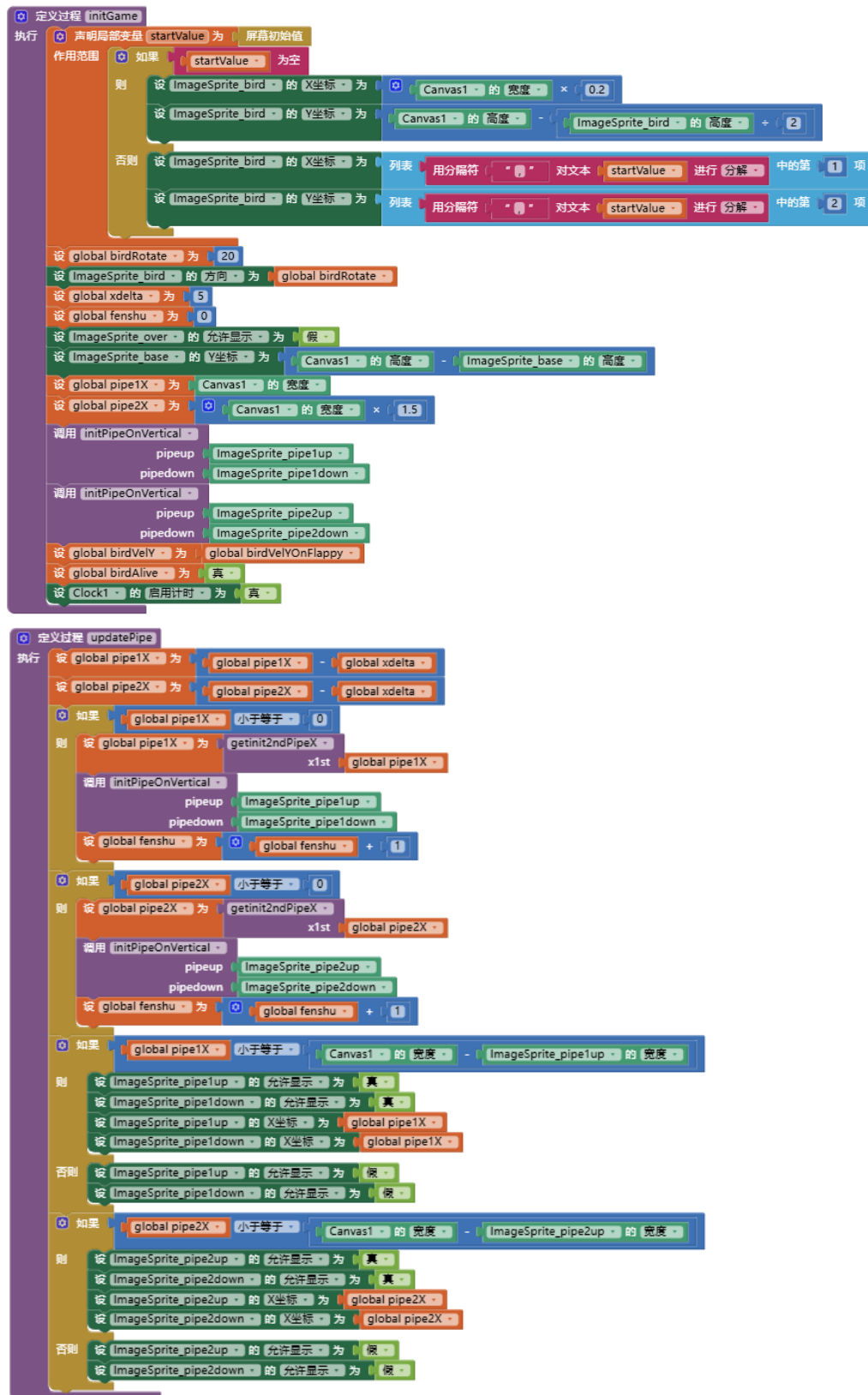
- 修改 Screen2 界面：
 - 再次上传柱子的图像素材文件;
 - 增加 2 个精灵表示柱子，设置图像、是否显示等属性，与之前类似;



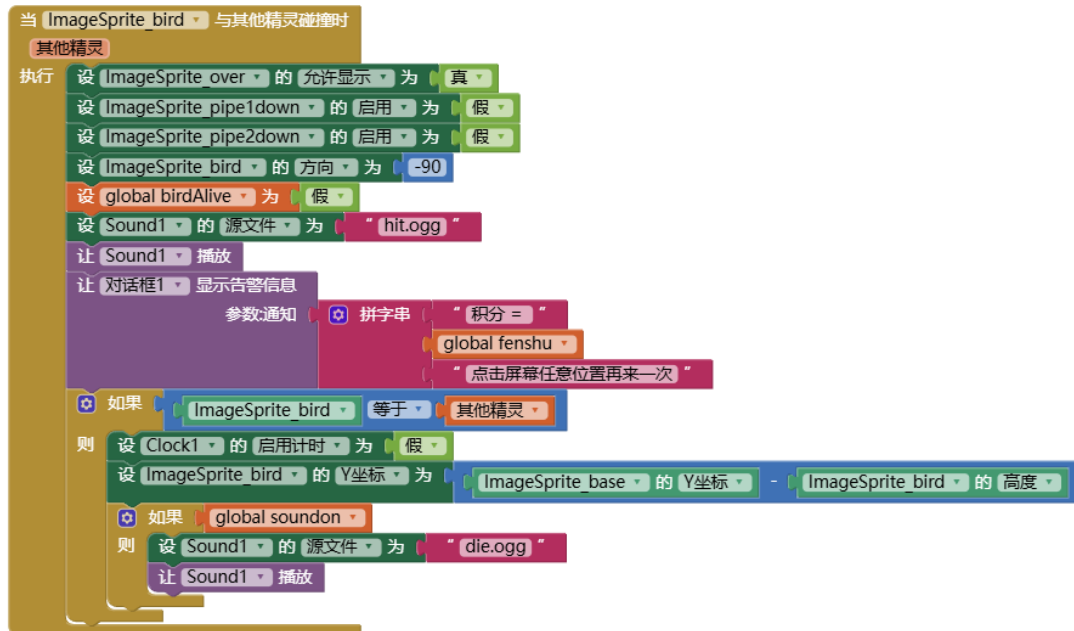
四、编写代码

- 修改屏幕 2 的初始化函数，计算随机生成的 2 组上下 4 根柱子的 XY 坐

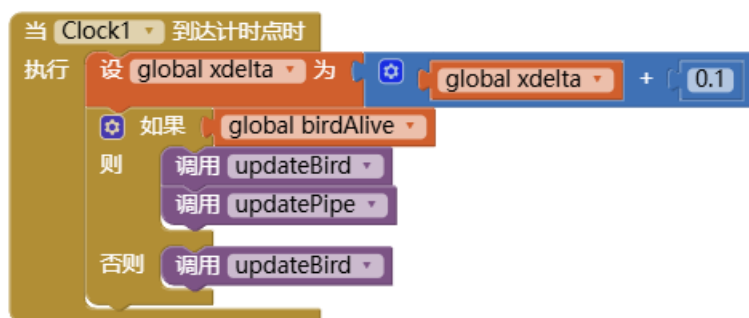
标和高度。因为开始时只显示一组柱子，当第一组柱子移动到差不多画布中央，第二组柱子才开始从右侧进入画布。增加的代码主要是初始化函数中增加队第二组柱子坐标的初始化和更新柱子。



- 在精灵的碰撞检测函数中增加柱子与小鸟碰撞的判断代码，如果发生碰撞则游戏失败，播放碰撞音效，小鸟继续自由下落，直到摔落到地基上，播放失败音效，停止游戏并显示游戏结束消息，结束消息中包括本地得分与最高得分，并提示用户再来一盘或者结束游戏。



- 使用数据库组件，由于数据库不能在不同屏幕间共享，重新开始游戏直接重新初始化 Screen 2，不返回 Screen1，记录最高得分。
- 拓展功能，随游戏进行难度提升（柱子移动速率加快）



- 拓展功能，暂停键和音量键；增添两个全局变量，表示是否暂停和是否开启声音，同时改变按钮的图像；暂停键启用时，将所有的 update 函数停止。

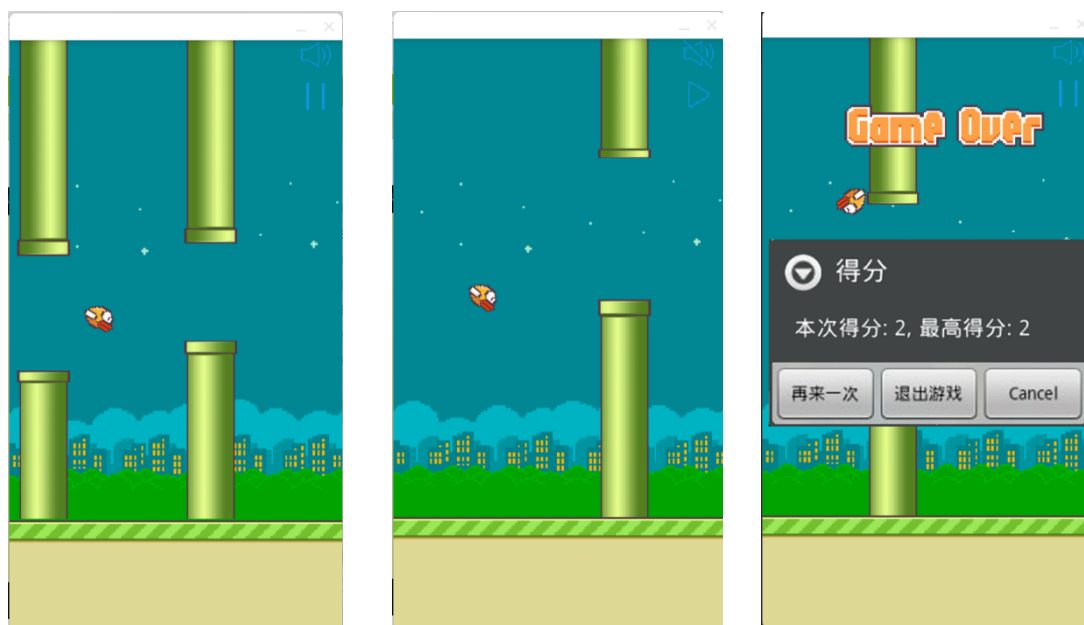
五、测试应用

- 打开 C:\CxcyCourse\AI2Companion_Win7_64bit\ailaunch.bat，启动桌面版 AI 伴侣；
- 在开发环境中，选择 Connect-》AI Companion，通过编码进行连接；

- 正确运行；

6 实验结果与分析

运行 AI 伴侣程序，开始游戏，随游戏进行，柱子移动变快变密，难度提升。重复几次游戏后，能正确记录最高得分。且右上角暂停键和音量键均能正确使用。



7 实验总结

除了按照实验手册完成基本实验内容，还可以通过以下方面对实验进行优化，如：

1. 合理确定 4 根柱子的随机数范围：柱子之间的 X 坐标应有固定的最小间距，避免生成过于密集的柱子导致难度过高。可以根据屏幕宽度和游戏节奏设定间隔；柱子上下之间的 Y 间隔应保持足够的空间，确保小鸟有机会通过。可以结合游戏难度动态调整。（已在上述代码块中体现）
2. 可以动态调整难度：随着游戏进展逐步缩短柱子的移动速度，水平间距或垂直间距，增加挑战性。还可以调整以下参数小鸟的初始速度和下落加速度等。（已在上述代码块中体现）
3. 增加暂停游戏和停止游戏声音功能：点击暂停图像后，停止游戏的更新循环；再次点击恢复更新。点击停止音乐图像后，控制背景音乐的播放与暂停状态。（已在上述代码块中体现）
4. 随机更换小鸟的图像和背景图片，让视觉体验更丰富

